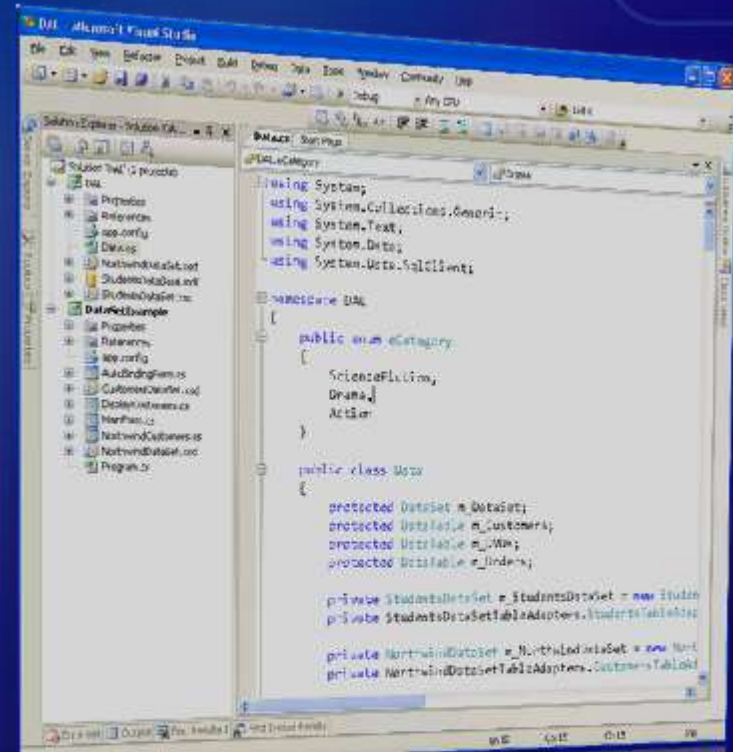
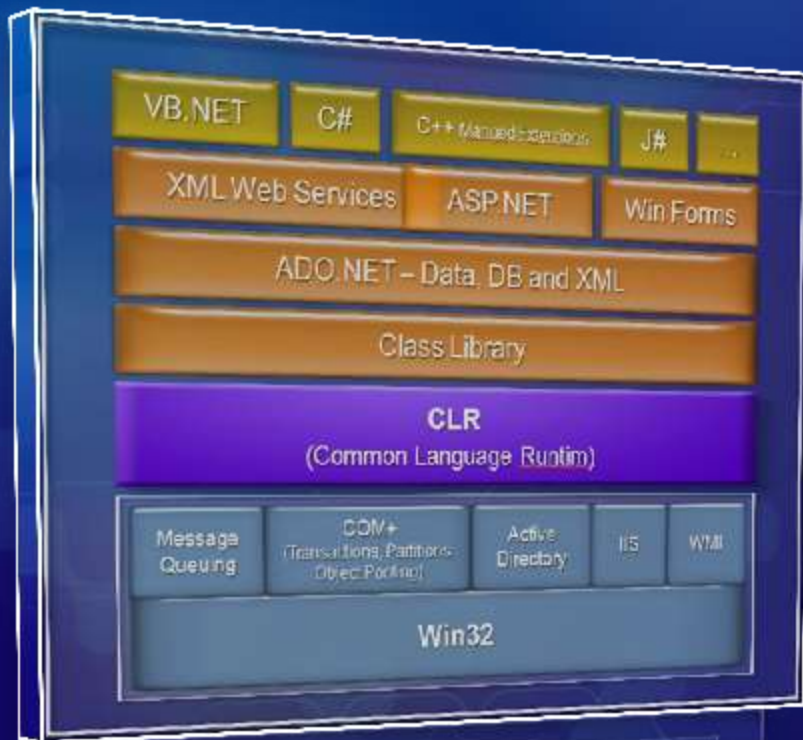


The .NET Platform

- The .NET Framework
- .NET Enterprise Servers
- .NET My Services
- Visual Studio .NET



The .NET Framework

VB.NET

C#

C++ Managed Extensions

J#

...

XML Web Services

ASP.NET

Win Forms

ADO.NET – Data, DB and XML

.NET Class Libraries and IDEs



CLR

(Common Language Runtime)

Message
Queuing

COM+
(Transactions, Partitions,
Object Pooling)

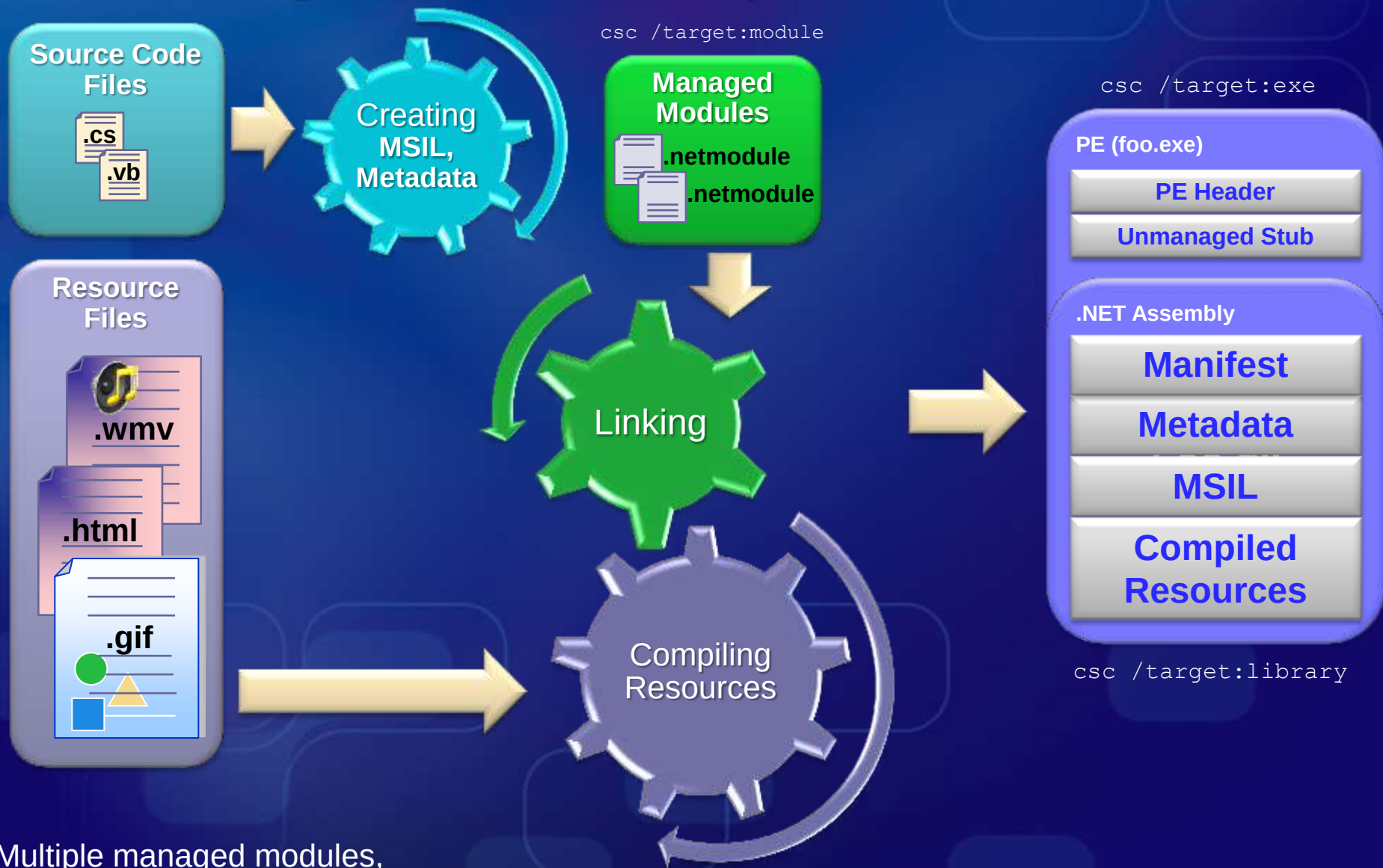
Active
Directory

IIS

WMI

Win32

Creating a .NET Assembly



Multiple managed modules, source code files and resource files are compiled to produce an assembly

Assembly Manifest

- **Name**

- **A string which is the name of the assembly**
by default: the same name of the file that contains it

- **Version**

- **A four-numbers field in the format of Mj.Mi.Rev.Build**

- **Major:**

Radical changes to the application/assembly which are visible and relevant to the user

(usually a major set of features that was added to the application and changes in its look-and-feel)

- **Minor:**

Minor changes to the application/assembly which are visible and relevant to the user

(usually a minor set of features that was added to the application with no changes in its look-and-feel)

- **Revision:**

Internal changes to the application/assembly which are not visible or directly relevant to the user

(usually some minor bug-fixes, security issues, performance improvements etc.)

- **Build:**

Every change in the source code, even if only semantic changes, requires a re-build.

(usually changes due to code review and code refactoring which does not influence any code logic)

Assembly Manifest

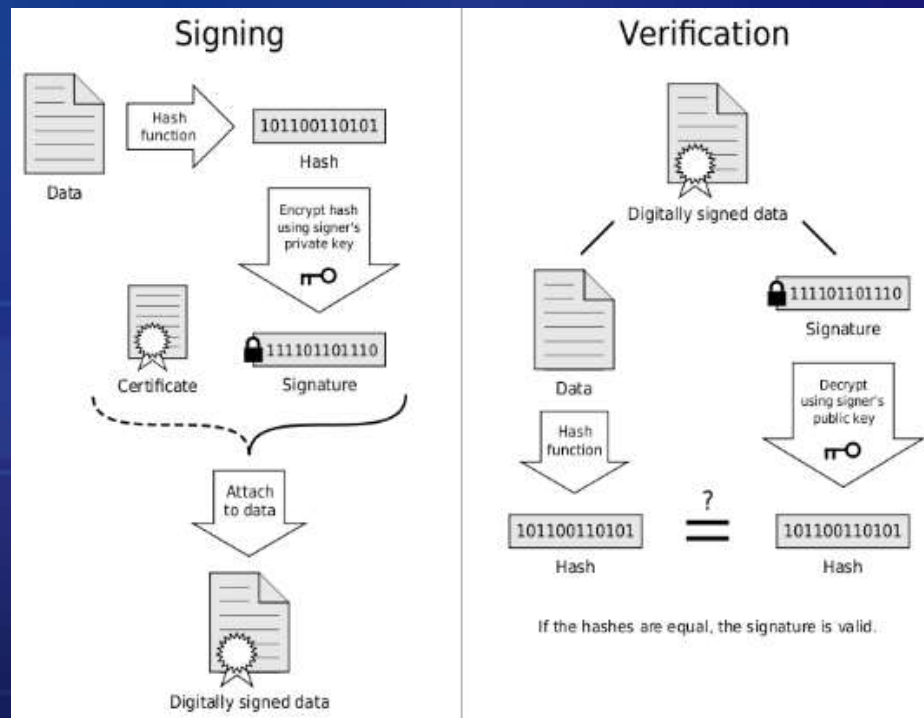
- **Locale**

- **A string which represents the localization code of the application/assembly**

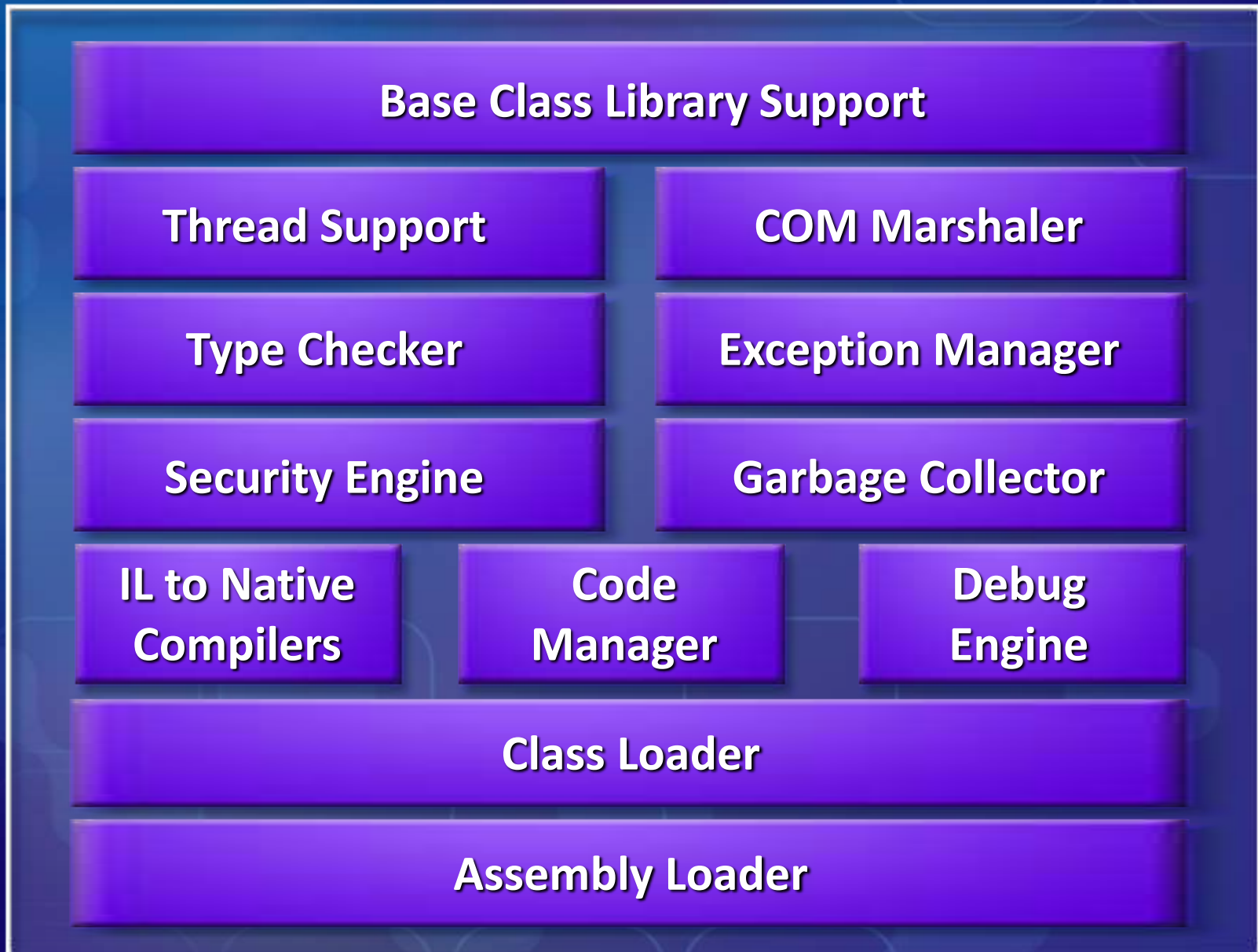
examples: en-US | he-IL | fr-FR
(defaults to null)

- **Public Key**

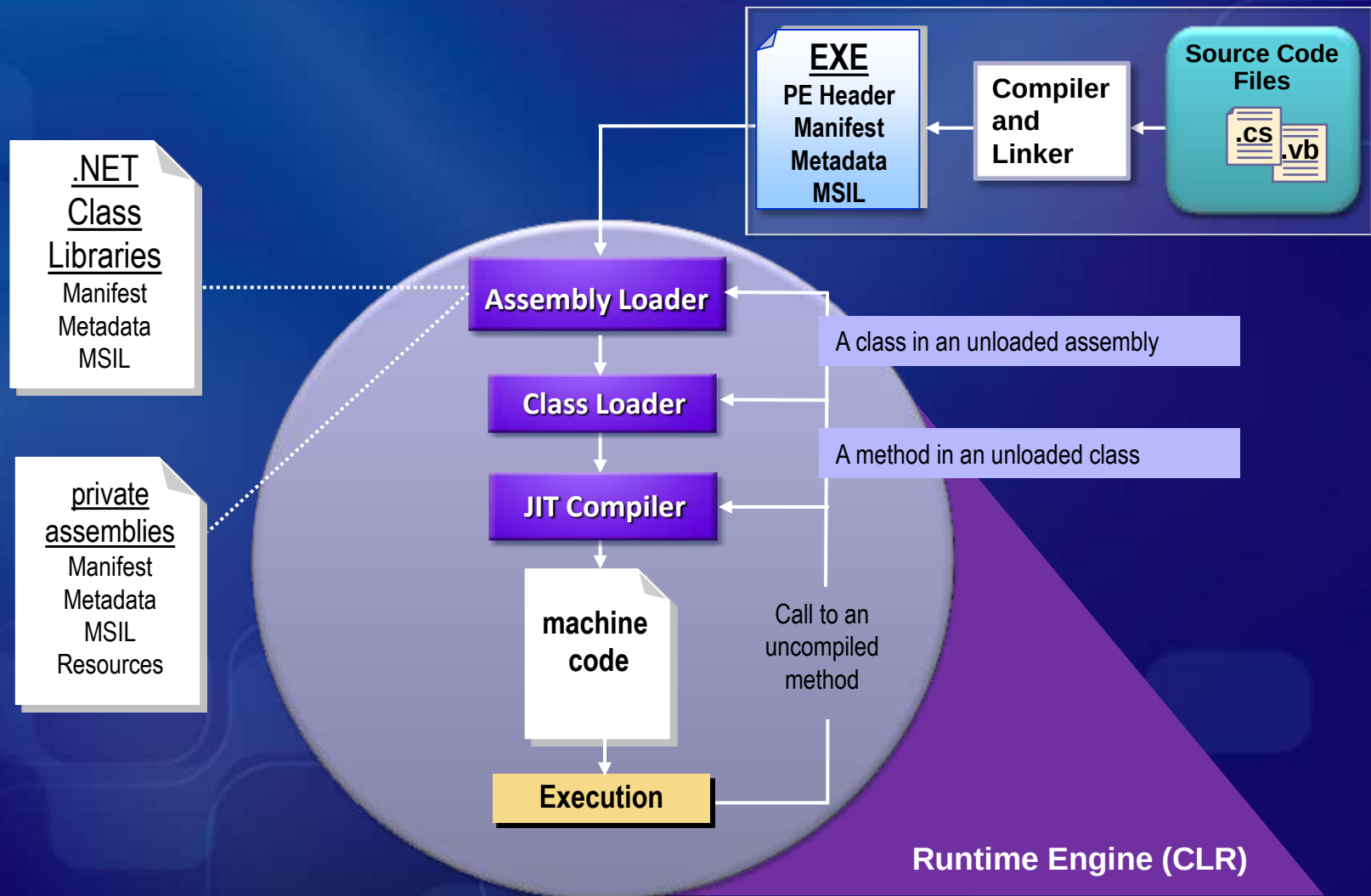
- **The public key of a digitally signed assembly**



The CLR (Common Language Runtime)



The Process of Managed Execution



Hello C#.NET

```
HelloCSharp.cs - Notepad
File Edit Format View Help
public class Program <-----
{
    static void Main() <-----
    {
        int count = 1;

        string userInput = System.Console.ReadLine(); <-----
        int.TryParse(userInput, out count); <-----
        for (int i = 0; i < count; ++i)
        {
            System.Console.WriteLine("Hello C#.NET!"); <-----
        }
    }
}
```

All code resides in classes

The entry point of the program

Use static methods of Console class for console I/O

Primitives are objects and have methods (i.e. count.ToString())

Syntax and statements are C/C++ like

```
Administrator: Visual Studio 2008 Command Prompt
Setting environment for using Microsoft Visual Studio 2008 x86 tools.
G:\Microsoft Visual Studio 9.0\VC>c:

C:\>csc HelloCSharp.cs
Microsoft (R) Visual C# 2008 Compiler version 3.5.30729.1
for Microsoft (R) .NET Framework version 3.5
Copyright (C) Microsoft Corporation. All rights reserved.

C:\>HelloCSharp.exe
3
Hello C#.NET!
Hello C#.NET!
Hello C#.NET!

C:\>_
```